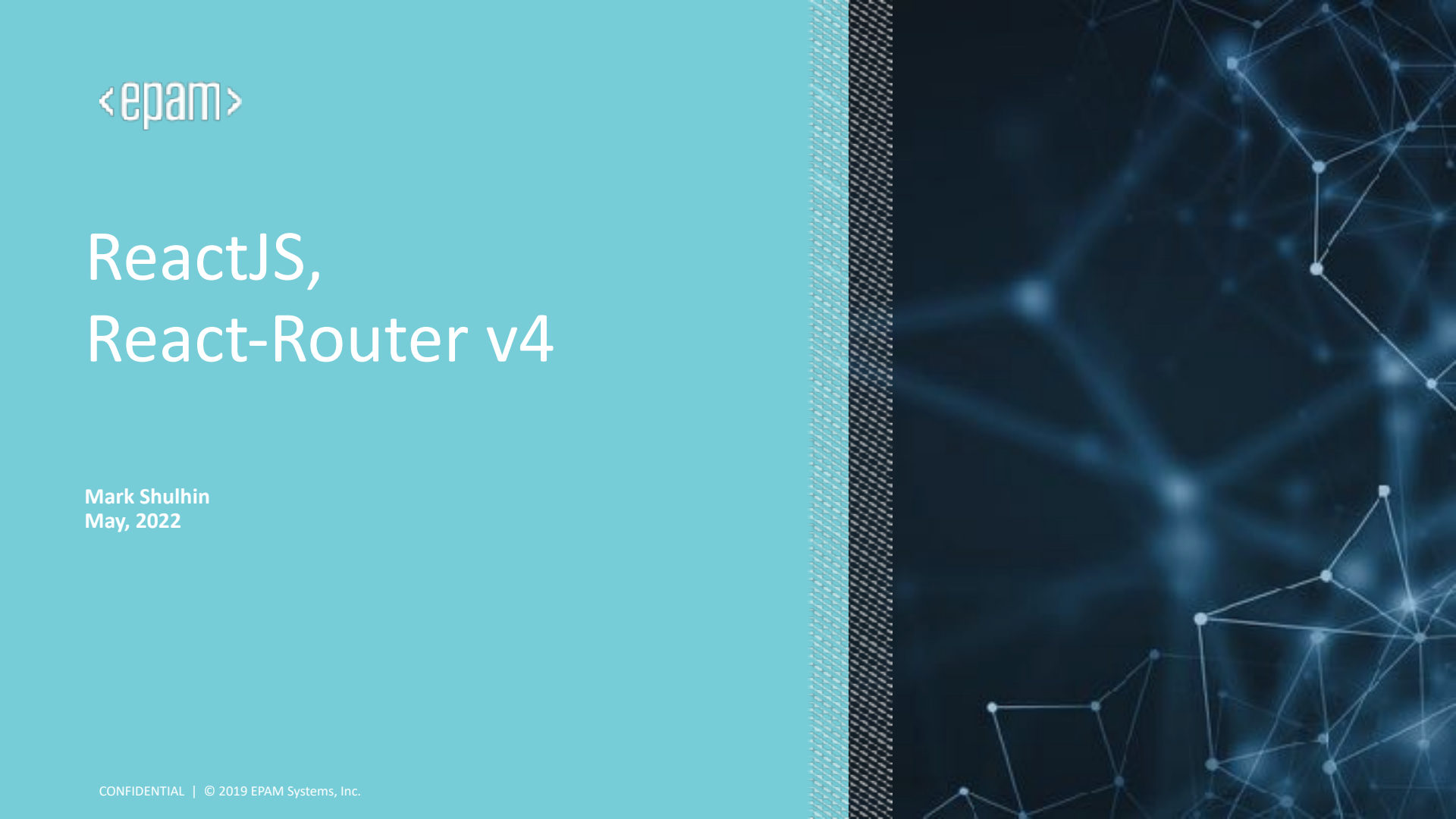


ReactJS, React-Router v4

Mark Shulhin
May, 2022

CONFIDENTIAL | © 2019 EPAM Systems, Inc.

What we will do today?

- React, React-Router 😯
- A lot of code 💻
- Ask questions

JSX in Depth

```jsx
import React from 'react';

// Wrong! This is a component and should have been capitalized:
function hello(props) {
  // Correct! This use of <div> is legitimate because div is a valid HTML tag:
  return <div>Hello {props.toWhat}</div>;
}

function HelloWorld() {
  // Wrong! React thinks <hello /> is an HTML tag because it's not capitalized:
  return <hello toWhat="World" />;
}
```

JSX in Depth(correct)

```jsx
import React from 'react';

// Correct! This is a component and should be capitalized:
function Hello(props) {
  // Correct! This use of <div> is legitimate because div is a valid HTML tag:
  return <div>Hello {props.toWhat}</div>;
}

function HelloWorld() {
  // Correct! React knows <Hello /> is a component because it's capitalized.
  return <Hello toWhat="World" />;
}
```

JavaScript Expressions as Props

You can pass any JavaScript expression as a prop, by surrounding it with {}. For example, in this JSX foo will evaluate to 10



Conditional Rendering

In React, you can create distinct components that encapsulate behavior you need. Then, you can render only some of them, depending on the state of your application.



Conditional Rendering

```
function Greeting(props) {
  const isLoggedIn = props.isLoggedIn;
  if (isLoggedIn) {
    return <UserGreeting />;
  }
  return <GuestGreeting />;
}

ReactDOM.render(
  // Try changing to isLoggedIn={true}:
  <Greeting isLoggedIn={false} />,
  document.getElementById('root')
);
```

Inline If-Else with Conditional Operator

```
render() {
  const isLoggedIn = this.state.isLoggedIn;
  return (
    <div>
      The user is <b>{isLoggedIn ? 'currently' : 'not'}</b> logged in.
    </div>
  );
}
```

What is component?

Components let you split the UI into independent, reusable pieces, and think about each piece in isolation



Ways of defining a React component?

- Functional component
- Extend React.Component
- Extend React.PureComponent

Functional

```
function FunctionalComponent1(props) {
    return <div>FunctionalComponent</div>;
}

const FunctionalComponent2 = props => {
    return <div>FunctionalComponent</div>;
};
```

React.Component

```
class ReactComponent extends React.Component {
  render() {
    return <div>ClassComponent</div>;
  }
}
```

React.PureComponent

```
class ReactPureComponent extends React.PureComponent {
  render() {
    return <div>ReactPureComponent</div>;
  }
}
```

Stateful Components vs. Stateless Co



```
class App extends React.Component {

  state = { count: 1 };

  handleCount(value) {
    this.setState((prevState) => (count: prevState.count + value }));
  }

  render () {
    return <div>hello</div>
  }

}
```

Stateful Components vs. Stateless Components

```
class Greeting extends React.Component {
  render () {
    return <h1>Hello, {this.props.name}</h1>;
  }
}

const AnotherGreeting = props => <h1>Hello {props.name}</h1>
```

Handling events

- React events are named using camelCase, rather than lowercase.
- With JSX you pass a function as the event handler, rather than a string.

Handling event in plain HTML

```
<button onClick="hackThePentagon()">
  Hack the pentagon
</button>
```

Handling event in React

```
<button onClick={hackThePentagon}>
  Hack the pentagon
</button>
```

Synthetic events

- Cross-browser wrapper around the browser’s native event
- It has the same interface as the browser’s native event, including stopPropagation() and preventDefault()
- Work identically across all browsers

Passing arguments to event handlers

```
<button onClick={(e) => this.deleteRow(id, e)}>Delete row</button>
<button onClick={this.deleteRow.bind(this, id)}>Delete row</button>
```

Binding functions in render

```
class Foo extends React.Component {
  handleClick() {
    console.log('clicked');
  }
  render () {
    return <button onClick={this.handleClick.bind(this)}>Click me</button>;
  }
}
```

Binding functions in render

```
class Foo extends React.Component {
  handleClick() {
    console.log('clicked');
  }
  render () {
    return <button onClick={() => this.handleClick()}>Click me</button>;
  }
}
```

Binding functions in constructor

```
class Foo extends React.Component {
  constructor(props) {
    super(props);
    this.handleClick = this.handleClick.bind(this);
  }
  handleClick() {
    console.log('clicked');
  }
  render () {
    return <button onClick={this.handleClick}>Click me</button>;
  }
}
```

Class properties proposal(experimental)



```
class Foo extends React.Component {
  handleClick = () => {
    console.log('clicked');
  }
  render () {
    return <button onClick={this.handleClick}>Click me</button>;
  }
}
```

Work with arrays

```
const numbers = [1, 2, 3, 4, 5];
const List = () => (
  <ul>
    {numbers.map(number =>
        <li>{number}</li>
    )}
  </ul>
);
```

Work with arrays: keys

```
const TodoList = () => (
  <ul>
    {todos.map(todo =>
      <li key={todo.id}>
        {todo.text}
      </li>
    )}
  </ul>
);
```

```jsx
function ListItem(props) {
  const value = props.value;
  return (
    // Wrong! There is no need to specify the key here:
    <li key={value.toString()}>
      {value}
    </li>
  );
}

function NumberList(props) {
  const numbers = props.numbers;
  const listItems = numbers.map((number) =>
    // Wrong! The key should have been specified here:
    <ListItem value={number} />
  );
  return (
    <ul>
      {listItems}
    </ul>
  );
}

const numbers = [1, 2, 3, 4, 5];
ReactDOM.render(
  <NumberList numbers={numbers} />,
  document.getElementById('root')
);
```

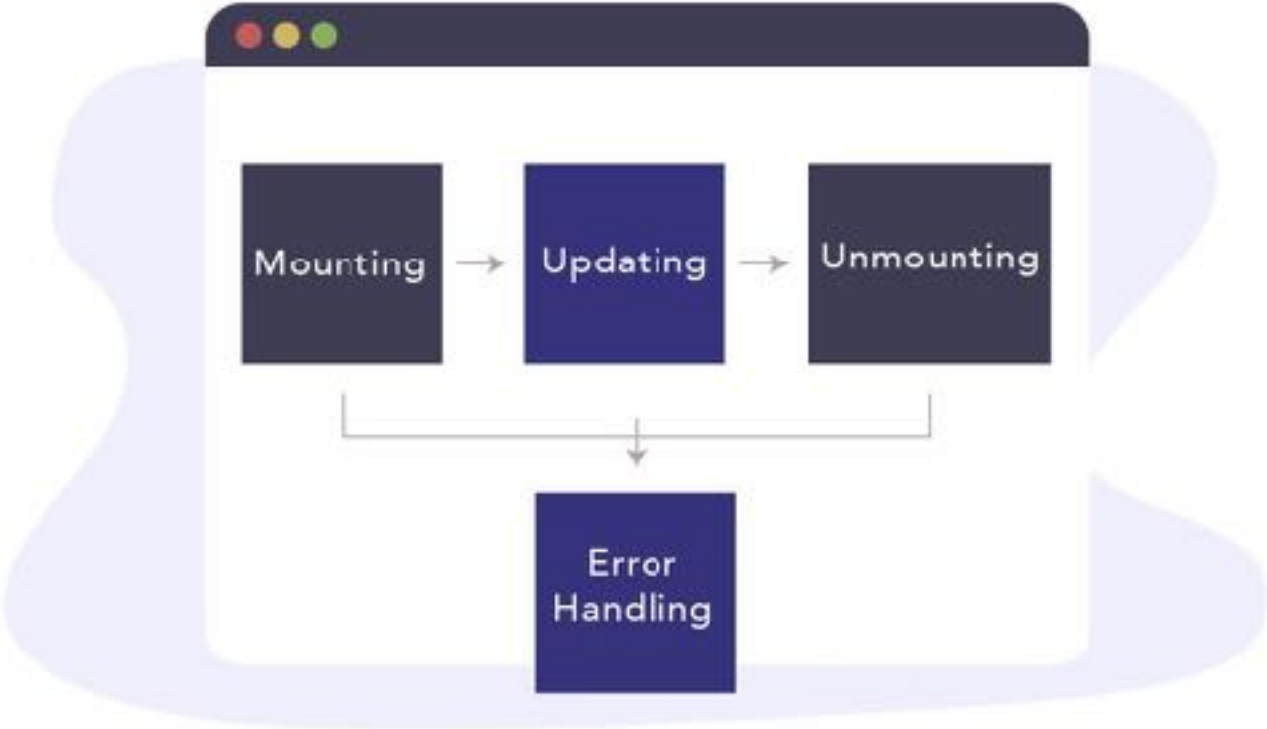
Returning multiple elements

```jsx
render() {
  return (
    <React.Fragment>
      <ChildA />
      <ChildB />
      <ChildC />
    <React.Fragment>
  );
}
```



```jsx
render() {
  return (
    <>
      <td>Hello</td>
      <td>World</td>
    </>
  );
}
```

Component lifecycle



Component lifecycle: Mounting

- `constructor()`
- `static getDerivedStateFromProps()`
- `render()`
- `componentDidMount()`

Component lifecycle: Updating

- `static getDerivedStateFromProps()`
- `shouldComponentUpdate()`
- **`render()`**
- `getSnapshotBeforeUpdate()`
- **`componentDidUpdate()`**

Component lifecycle: Unmounting

- `componentWillUnmount()`

Component lifecycle: Error handling

- `static getDerivedStateFromError()`

`componentDidCatch()`

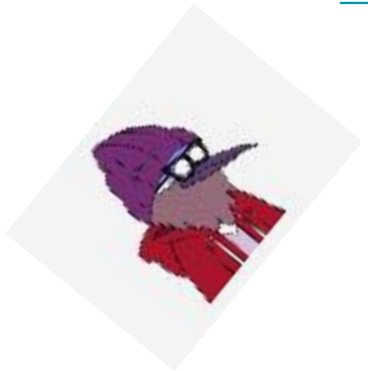
Lifecycle hooks diagram





http://projects.wojtekmaj.pl/react-lifecycle-methods-diagram/

https://wavez.github.io/react-hooks-lifecycle/







React.Component vs React.PureComponent

PureComponent is exactly the same as Component except that it handles the shouldComponentUpdate method for you

Component

```
shouldComponentUpdate(nextProps, nextState) {
  return true;
}
```

PureComponent

```
shouldComponentUpdate(nextProps, nextState) {
  return !shallowEqual(nextProps, this.props)
      || !shallowEqual(nextState, this.state);
}
```

Refs

Refs provide a way to access DOM nodes or React elements created in the render method.

There are a few good use cases for refs:

- Managing focus, text selection, or media playback.
- Triggering imperative animations.
- Integrating with third-party DOM libraries.

Creating refs

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.myRef = React.createRef();
  }
  render() {
    return <div ref={this.myRef} />;
  }
}
```

Accessing Refs

When a ref is passed to an element in render, a reference to the node becomes accessible at the current attribute of the ref.

```
const node = this.myRef.current;
```

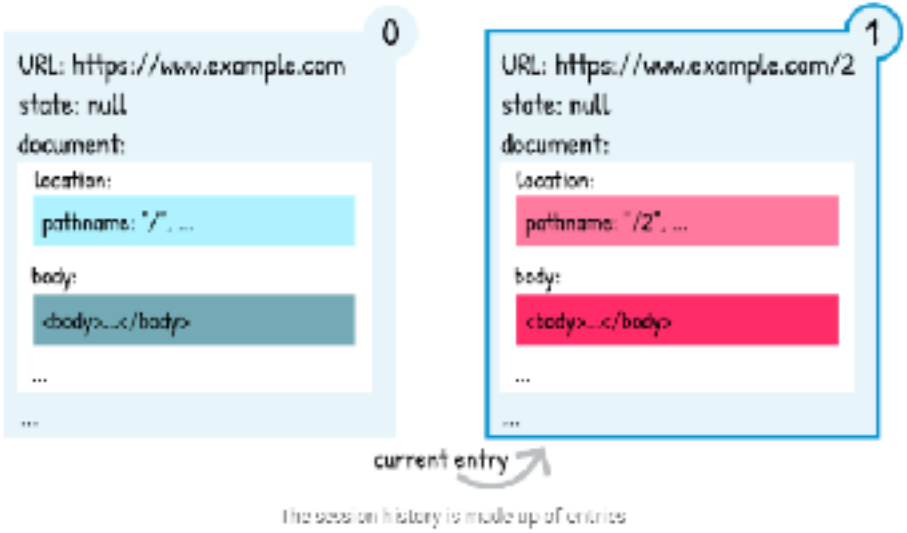


REACT / ROUTER

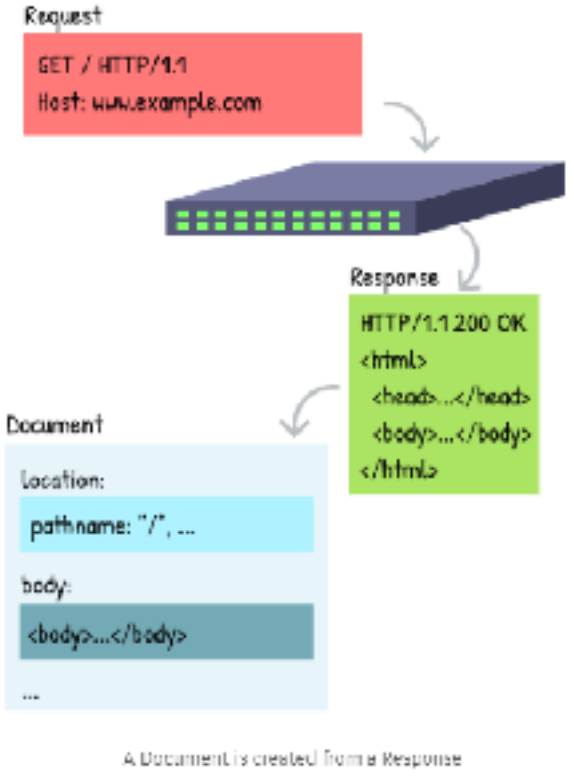
Web apps routing approaches

- Sending request to a server for a new page on each route change
- Single page application(SPA)

New document on each request



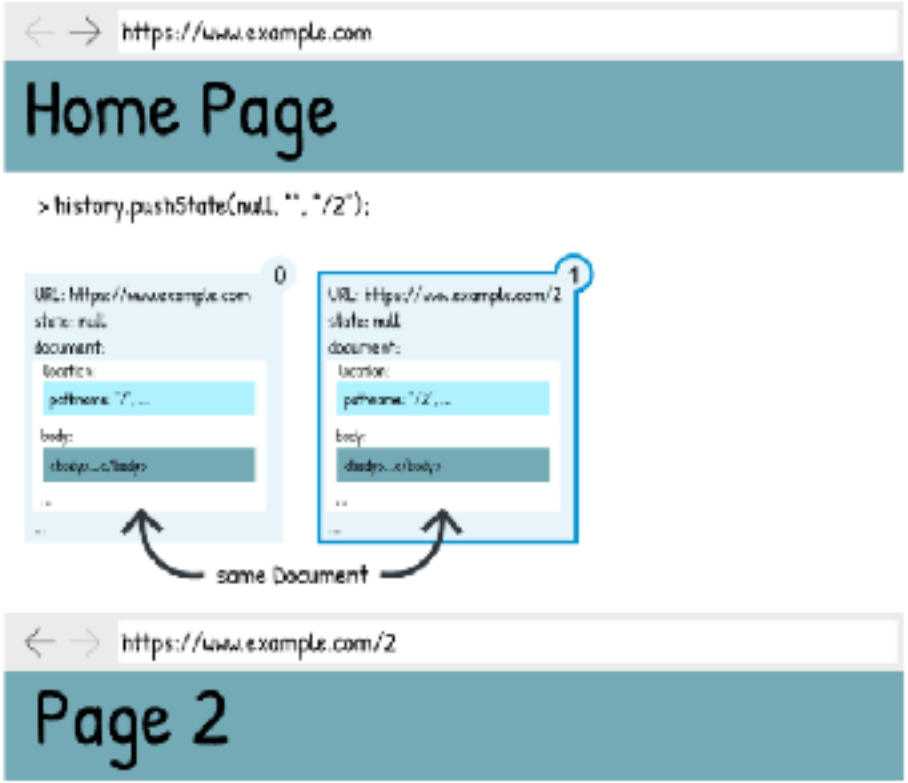
The session history is made up of entries



A Document is created from a Response

Single page application

A single-page application (SPA) is a website that re-renders its content in response to navigation actions (e.g. clicking a link) without making a request to the server to fetch new HTML.



React Router v4

React Router v4 is a pure React rewrite of the popular React package. Previous versions of React Router used configuration disguised as pseudo-components and could be difficult to understand. With v4, everything is “just components”.

Package	Version	Docs	Description
`react-router`	npm v4.3.1	api docs site api docs markdown	The core of React Router
`react-router-dom`	npm v4.3.1	api docs site api docs markdown	DOM bindings for React Router
`react-router-native`	npm v4.3.0	api docs site api docs markdown	React Native bindings for React Router
`react-router-config`	npm v1.0.0-beta.4	api docs readme	Static route config helpers

Installation

Use npm or yarn to add the package to your project



Routers

At the core of every React Router application should be a router component. For web projects, react-router-dom provides <BrowserRouter> and <HashRouter> routers



Defining routes

Route matching is done by comparing a <Route>'s path prop to the current location’s pathname. When a <Route> matches it will render its content and when it does not match, it will render null. A <Route> with no path will always match.



```
// when location = { pathname: '/about' }
<Route path="about" component={About} /> // renders <About />
<Route path="contact" component={Contact} /> // renders null
<Route component={Always} /> // renders <Always />
```

Grouping routes

The <Switch> is not required for grouping <Route>s, but it can be quite useful. A <Switch> will iterate over all of its children <Route> elements and only render the first one that matches the current location.



```
<Switch>
  <Route exact path="/" component={Home} />
  <Route path="/about" component={About} />
  <Route path="contact" component={Contact} />
  {/* When none of the above match, <NotFoundPage /> will be rendered */}
  <Route component={NotFoundPage} />
</Switch>
```

Navigation

React Router provides a <Link> component to create links in your application. Wherever you render a <Link>, an anchor (<a>) will be rendered in your application's HTML.

```
<Link to="/">Home</Link>
// <a href="/">Home</a>
```

Styling active link

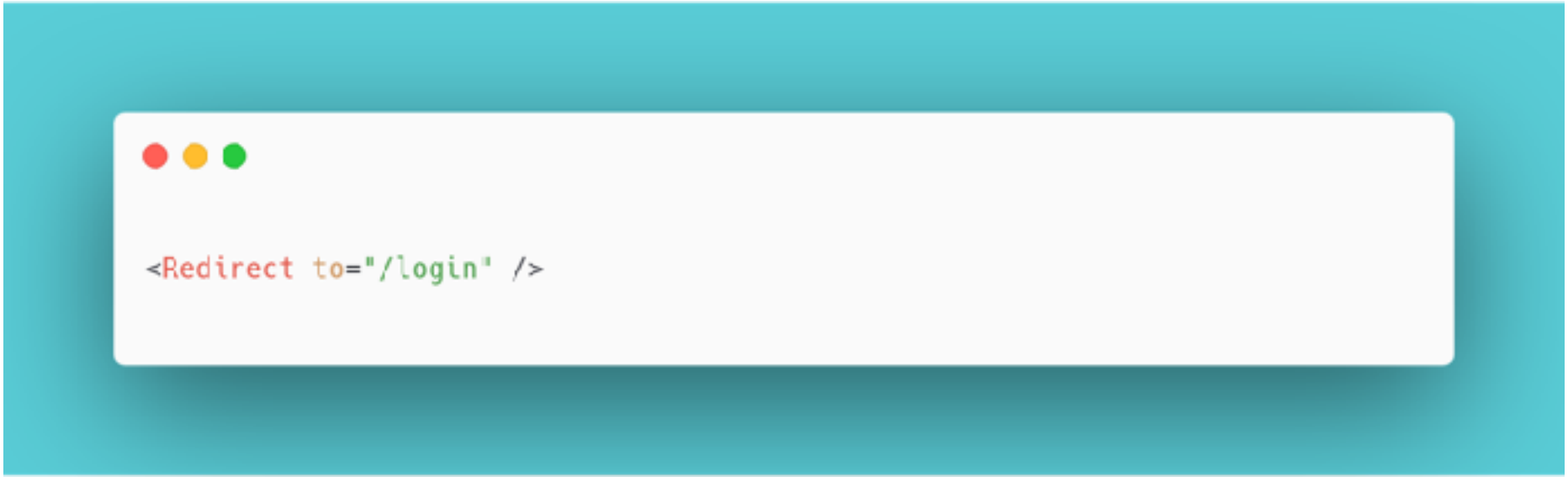
The <NavLink> is a special type of <Link> that can style itself as “active” when its to prop matches the current location.

```
// location = { pathname: '/about' }
<NavLink to="/about">About us</NavLink> // will have class 'active' when matches location
// <a href="/about" className="active">About us</a>

// Custom active class name
<NavLink to="/about" activeClassName="current-link">About us</NavLink>
// <a href="/about" className="current-link">About us</a>
```

Redirect to the other page

Any time that you want to force navigation, you can render a <Redirect>. When a <Redirect> renders, it will navigate using its to prop.



Prompt action before leaving the page

When your application enters a state that should prevent the user from navigating away (like a form is half-filled out), render a <Prompt>.

```
import { Prompt } from 'react-router-dom';

<Prompt
  when={formIsPartiallyFilled}
  message="Are you sure you want to leave?"
/>
```

Thank you!